

Web Forms

JSR-303 (bean validation) es el standard de Java para validación de POJOs, que nos permite por medio de anotaciones, establecer requisitos a ser aplicados sobre distintos atributos. Como todo JSR, el standard define una serie de interfaces y anotaciones bajo el paquete `javax` y existen múltiples implementaciones de la misma.

Para usarla, por tanto, debemos agregar dos dependencias nuevas al pom padre. La que define el JSR-303, y la implementación concreta que queremos usar. En este caso vamos a usar una implementación hecha por la propia gente de Hibernate, aunque su uso es independiente del uso de Hibernate como ORM en la capa de persistencia.

Las dos dependencias a agregar en el pom padre son:

```
<javax.validation-api.version>2.0.1.Final</javax.validation-api.version>
<org.hibernate.validator>6.2.4.Final</org.hibernate.validator>
```

```
<dependency>
  <groupId>javax.validation</groupId>
  <artifactId>validation-api</artifactId>
  <version>${javax.validation-api.version}</version>
</dependency>
<dependency>
  <groupId>org.hibernate</groupId>
  <artifactId>hibernate-validator</artifactId>
  <version>${org.hibernate.validator}</version>
</dependency>
```

Luego, en el pom de webapp, ya que es en esta capa que vamos relizar las validaciones y a presentar los errores, declaramos las mismas dependencias:

```
<dependency>
  <groupId>javax.validation</groupId>
  <artifactId>validation-api</artifactId>
</dependency>
<dependency>
  <groupId>org.hibernate</groupId>
  <artifactId>hibernate-validator</artifactId>
</dependency>
```

Debemos ahora definir el objeto a ser validado. Para esto vamos a crear un modelo específico para este fin, que represente al formulario a ser validado. Este modelo es claramente propio de la vista a la que corresponde, por lo que vamos a incluirlo en el módulo de webapp, bajo el paquete `ar.edu.itba.webapp.form`.

```
public class UserForm {

    @Size(min = 6, max = 100)
    @Pattern(regexp = "[a-zA-Z0-9]+")
    private String username;

    @Size(min = 6, max = 100)
    private String password;
```

```

@Size(min = 6, max = 100)
private String repeatPassword;

public String getUsername() {
    return username;
}

public void setUsername(String username) {
    this.username = username;
}

public String getPassword() {
    return password;
}

public void setPassword(String password) {
    this.password = password;
}

public String getRepeatPassword() {
    return repeatPassword;
}

public void setRepeatPassword(String repeatPassword) {
    this.repeatPassword = repeatPassword;
}
}

```

y en el controller agregamos nuestros nuevos endpoints / modificamos los anteriores:

```

@RequestMapping("/")
public ModelAndView index(@ModelAttribute("registerForm") final UserForm form) {
    return new ModelAndView("index");
}

@RequestMapping(value = "/create", method = { RequestMethod.POST })
public ModelAndView create(@Valid @ModelAttribute("registerForm") final UserForm
form, final BindingResult errors) {
    if (errors.hasErrors()) {
        return index(form);
    }

    final User u = us.create(form.getUsername(), form.getPassword());
    return new ModelAndView("redirect:/user?userId=" + u.getId());
}

```

Notese el annotation `@Valid` que le pide a spring que valide el formulario antes de llamar a nuestro controller. Sobre el objeto `BindingResult` está presente la información sobre los errores encontrados.

Respecto de la vista, el form está attacheado a la vista gracias al annotation `@ModelAttribute` bajo el nombre `registerForm`, y para accederlo facilmente,

utilizaremos una taglib provista por el propio Spring para formularios.

```
<%@ taglib prefix="c" uri="http://java.sun.com/jstl/core_rt"%>
<%@ taglib prefix="form" uri="http://www.springframework.org/tags/form" %>

<html>
<head>
  <link rel="stylesheet" href="<c:url value="/css/style.css"/>" />
</head>
<body>
<h2>Register</h2>
<c:url value="/create" var="postPath"/>
<form:form modelAttribute="registerForm" action="${postPath}" method="post">
  <div>
    <form:label path="username">Username: </form:label>
    <form:input type="text" path="username"/>
    <form:errors path="username" cssClass="formError" element="p"/>
  </div>
  <div>
    <form:label path="password">Password: </form:label>
    <form:input type="password" path="password" />
    <form:errors path="password" cssClass="formError" element="p"/>
  </div>
  <div>
    <form:label path="repeatPassword">Repeat password: </form:label>
    <form:input type="password" path="repeatPassword"/>
    <form:errors path="repeatPassword" cssClass="formError" element="p"/>
  </div>
  <div>
    <input type="submit" value="Register!" />
  </div>
</form:form>
</body>
</html>
```

El tag `<form:errors/>` permite customizar el tipo de elemento HTML a usar, su clase CSS, su estilo inline, etc.

Vimos que los mensajes que aparecen son los defaults y no están muy acordes a lo que nos gustaría mostrarle al usuario. Para esto debemos crear nuestro propio message source y escribir nuestros propios mensajes.

Para esto, en `WebConfig` agregamos un nuevo bean:

```
@Bean
public MessageSource messageSource() {
    final ReloadableResourceBundleMessageSource messageSource = new
ReloadableResourceBundleMessageSource();
    messageSource.setBasename("classpath:i18n/messages");
    messageSource.setDefaultEncoding(StandardCharsets.UTF_8.displayName());
    messageSource.setCacheSeconds(5);
    return messageSource;
}
```

Y en `src/main/resources` creamos un directorio `i18n` donde escribiremos nuestros mensajes. El archivo default es, siguiendo el `basename` que se ha configurado, con la extensión `.properties`. En el medio, entre extensión y `basename`, va la información del locale sobre el que aplica dicho archivo, sea por idioma o región, separados por guiones bajos. Así:

1. `messages.properties` es el archivo con mensajes default
2. `messages_en.properties` es el archivo con mensajes para idioma inglés
3. `messages_en_US.properties` es el archivo con mensajes para idioma inglés de estados unidos

En todos los casos, al querer obtener un mensaje, se buscará en el archivo más específico posible. Si el archivo no existe o no contiene el mensaje, la búsqueda continua en un archivo menos específico (por ejemplo, por idioma, sin región), y finalmente caerá en el archivo default.

Escribimos entonces nuestro `messages.properties`

```
Size=El nombre de usuario debe ser entre {2} y {1} caracteres
Pattern.UserForm.username=El campo no matchea mi criterio arbitrario
```

En el primer caso, estamos definiendo un mensaje default para todos los usos del annotation `@Size`. En el segundo, estamos definiendo un mensaje particular para el annotation `@Pattern` aplicado sobre la clase `UserForm` en el campo `username`.

Adicionalmente, podemos aprovechar para internacionalizar los otros mensajes que estaban en JSPs.

Para esto modificamos `users.jsp` y usamos otra taglib de spring para usar mensajes internacionalizados.

```
<%@ taglib prefix="c" uri="http://java.sun.com/jstl/core_rt"%>
<%@ taglib prefix="spring" uri="http://www.springframework.org/tags"%>

<html>
<head>
  <link rel="stylesheet" href="<c:url value="/css/style.css"/>" />
</head>
<body>
<h2><spring:message code="user.greeting" arguments="{user.username}"/></h2>
<h5><spring:message code="user.id" arguments="{user.id}"/></h5>
</body>
</html>
```

y agregamos en el `messages.properties` los nuevos mensajes:

```
user.id={0} your current id is! - Master Yoda
user.greeting=Hello {0}!
```

Por su parte, vimos que el annotation `@ModelAttribute`, que bindea un objeto como atributo de un modelo, puede ser usado de otras formas. Anotando un método de un controller, spring llamará automáticamente a dicho método por cada request mapeado en el mismo, agregando el modelo correspondiente. Caso de uso típico de esto, es el adjuntar información del usuario logueado a cada request. Por ejemplo, para hacer esto agregamos un método al controller:

```
@ModelAttribute("userId")
public Integer loggedUser(final HttpSession session) {
    return (Integer) session.getAttribute(LOGGED_USER_ID);
}
```